

# PhoenixGuard Complete System Blueprint

Professional end-to-end blueprint for the PhoenixGuard V3 live chart-intelligence, Model Council, dashboard, and calibrated execution system.

|                     |                                                    |
|---------------------|----------------------------------------------------|
| Generated           | 2026-06-11                                         |
| Canonical runtime   | PhoenixGuard V3 / FINAL_LIVE                       |
| Production launcher | launch_phoenixguard_live_ready.ps1                 |
| Primary authority   | PG_EXECUTION_PACKET_V3                             |
| Rendered source     | docs\architecture\PhoenixGuard_System_Blueprint.md |

Design principle: observation is not execution; study is not execution; only a fresh, validated V3 execution packet can reach calibrated broker actions.

# Contents

|                                                 |  |
|-------------------------------------------------|--|
| Executive Summary                               |  |
| What The System Does                            |  |
| Why It Is Built This Way                        |  |
| Canonical Runtime Chain                         |  |
| High-Level Architecture Map                     |  |
| Primary Dataflow                                |  |
| Production Launch Setup                         |  |
| Repository Structure                            |  |
| Core Runtime Concepts                           |  |
| Observation, Study, Execution                   |  |
| State Advancement                               |  |
| Side Semantics                                  |  |
| Capture And Window Tracker Design               |  |
| Tracker Techniques Used                         |  |
| Vision Layer                                    |  |
| Vision Techniques Used                          |  |
| Market Object Registry And Overlay Architecture |  |
| Memory System                                   |  |
| Decision And Reasoning Layer                    |  |
| Model Council V3                                |  |
| Skill Gates                                     |  |
| Scenario And Simulation Reasoning               |  |
| Execution Packet V3 Contract                    |  |
| Shooter And Broker Action Design                |  |
| Calibrated Targets                              |  |
| Shooter Techniques Used                         |  |
| API And Dashboard Layer                         |  |
| Mobile Observer                                 |  |
| Voice Control                                   |  |
| Training And Local Model Runtime                |  |
| Continual Adaptation And Adapters               |  |
| Runtime Integrity And Observability             |  |
| Security And State Protection                   |  |
| Simulation, Backtesting, And Replay             |  |
| Testing And Certification Matrix                |  |
| Deployment And Sharing                          |  |

Important Data Artifacts  
Operator Safety Rules  
Failure Handling And Diagnostics  
Design Principles Used  
End-To-End Build Explanation  
Extension Points  
Glossary  
Final Blueprint Summary

Generated: 2026-06-11

Runtime profile: PhoenixGuard V3 / FINAL\_LIVE

Primary launcher: launch\_phoenixguard\_live\_ready.ps1

Primary execution authority: PG\_EXECUTION\_PACKET\_V3

Document source: repository code, V3 language constitution, runtime maps, API routes, tests, launch scripts, and architecture documents present in this workspace.

## Executive Summary

PhoenixGuard is a local-first chart intelligence and execution-control workstation. It watches a locked broker or study surface, extracts chart state from live screenshots, evaluates that state through multiple reasoning and model layers, publishes non-executable study packets while a setup is forming, and permits broker interaction only when a validated V3 execution packet passes runtime, model, timing, instrument, calibration, and shooter gates.

The system is built around one strict doctrine: observation is not execution. Raw BUY or SELL signals, legacy action fields, old skill gate outputs, memory confidence, dashboard displays, and study packets are all diagnostic or advisory. The only valid path into live broker action is a fresh PG\_EXECUTION\_PACKET\_V3 consumed by shooter .py and executed through ShooterActionSequencerV2 using calibrated screen targets.

The architecture is intentionally layered. The live tracker produces state. Vision modules and chart transforms produce structure. Memory and simulation modules provide historical and synthetic context. Decision modules score market reality, price location, regimes, scenarios, timing, and model-council maturity. Execution modules validate the packet contract and preserve the broker amount. Runtime modules provide cache integrity, observability, model warm-state checks, instrument locks, and certification. The FastAPI mobile API exposes dashboards, state, packets, health, and control endpoints.

## What The System Does

PhoenixGuard performs these jobs as one coordinated system:

- 1 Captures a live broker or chart window and keeps that source locked to avoid wrong-surface execution.
- 1 Converts frames into structured chart state: candles, zones, support/resistance, projections, smart-money context, overlay objects, timing information, and sequence context.
- 1 Runs local and optional model-backed analysis over the chart image and derived state.
- 1 Uses memory retrieval and replay style features to compare current market behavior to previous setups.
- 1 Scores market reality, trap risk, entry quality, price location, regime, continuation probability, and target-before-invalidation probabilities.
- 1 Promotes a setup through Model Council stages from observation to executable packet.
- 1 Publishes study packets for visibility while a setup is not executable.
- 1 Publishes a V3 execution packet only when the council and runtime contract agree.
- 1 Uses a separate shooter process to validate the packet again, set expiry/time, preserve amount, and click only BUY or SELL.
- 1 Serves live dashboard, floating-state, health, trace, and packet endpoints through FastAPI.
- 1 Records telemetry, visual evidence, paper/live decisions, replay data, and test reports for auditability.

# Why It Is Built This Way

The project handles a high-risk workflow: a live visual system can ultimately interact with a broker UI. A direct model-to-click design would be fragile because visual inference can be stale, noisy, on the wrong window, or internally contradictory. PhoenixGuard therefore uses a separation-of-authority architecture:

- 1 Perception can observe but cannot execute.
- 1 Dashboard state can explain but cannot execute.
- 1 Study packets can describe a forming setup but cannot execute.
- 1 Model Council can arbitrate but its final\_side alone cannot execute.
- 1 The V3 packet schema can authorize only after freshness, identity, model health, timing, and side agreement pass.
- 1 The shooter performs a second live read, validates discipline and calibration, and rechecks before the side click.

This design reduces accidental actions, stale clicks, wrong-window clicks, amount changes, and legacy-path execution. It also makes debugging precise because every stage emits a traceable artifact.

# Canonical Runtime Chain

The V3 language constitution defines the canonical chain:

```
V3 Launcher
-> Tracker / Frame Capture
-> Vision Models
-> Model Council V3
-> Market Reality Engine
-> Execution Lane Resolver
-> Timing / Path-Aware Timing Engine
-> Study Packet Publisher
-> Execution Packet Publisher
-> PG_EXECUTION_PACKET_V3 Validator
-> ShooterActionSequencerV2
-> Calibrated Time / Side Action
-> FloatingStateV2
-> Observability / Runtime Trace
```

Everything outside this chain is support, diagnostics, legacy compatibility, testing, training, replay, or deployment assistance.

# High-Level Architecture Map

| Layer               | Core Files                                                                                                | Responsibility                                                                                                                      |
|---------------------|-----------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------|
| Launch and profile  | launch_phoenixguard_live_ready.ps1, start_phoenixguard_full_local.ps1, start_phoenixguard_24_7_tracker.py | Starts the production live stack, pins the FINAL_LIVE profile, prepares runtime state, and launches API/tracker/shooter components. |
| Capture and tracker | phoenixguard/mobile_api/window_tracker.py                                                                 | Locks a window, captures frames, builds live session state, derives chart structure, publishes artifacts, and feeds Model Council.  |

| Layer                     | Core Files                                                                                                   | Responsibility                                                                                                                                              |
|---------------------------|--------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Vision and overlay        | phoenixguard/vision/*, phoenixguard/tracking/market_object_tracker_v3.py                                     | Performs preprocessing, source-lock checks, chart transforms, object tracking, overlays, layer management, visual health, and registry persistence.         |
| Memory                    | phoenixguard/memory/*                                                                                        | Embeds and retrieves prior examples, scores visual-play memory confirmation, and produces sequence/style similarity features.                               |
| Decision                  | phoenixguard/decision/*                                                                                      | Performs regime, market reality, price location, scenarios, skill-gate diagnostics, RL/regression, model-council arbitration, and timing readiness scoring. |
| Execution contract        | phoenixguard/execution/packet_v3.py, phoenixguard/execution/v3_language.py, phoenixguard/runtime/cache_v3.py | Defines canonical schema vocabulary, packet construction, validation, cache integrity, TTL, side agreement, sequence context, and runtime integrity.        |
| Shooter                   | shooter.py, phoenixguard/execution/shooter_action_sequencer.py, phoenixguard/execution/shooter_modes.py      | Reads only V3 packets, validates shooter gates, preserves visible amount, sets expiry/time, clicks calibrated BUY/SELL targets, and records evidence.       |
| API and dashboard         | phoenixguard/mobile_api/app.py, phoenixguard/mobile_api/static/window_tracker_dashboard.html, assets/js/*    | Exposes health, live state, packet, tracker, floating-state, artifact, registry, visual, dashboard, stream, and control endpoints.                          |
| Runtime and observability | phoenixguard/runtime/*, phoenixguard/tracing.py, tools/*                                                     | Provides model warm-state, cache validation, telemetry, atomic writes, freshness, certification, traces, and diagnostics.                                   |
| Simulation and training   | phoenixguard/simulation/*, phoenixguard/training/*, train_*.py, scripts/*                                    | Creates synthetic and replay scenarios, paper execution, event backtests, LoRA/adapters, clean splits, sequence manifests, and model exports.               |
| Mobile and voice          | mobile/android/*, phoenixguard/voice/*                                                                       | Provides Android observer UI and voice-command control layers for tracker status, capture, start/stop, interval updates, and command routing.               |

## Primary Dataflow

The live system has one preferred dataflow:

- 1 The launcher starts the mobile API, tracker, optional model-council daemon, dashboard, and shooter.
- 2 The tracker locks the target window and captures a frame.
- 3 The tracker validates the captured surface and derives chart geometry, candle rows, overlays, zones, projections, market object registry, and timing state.
- 4 Vision and tracking modules enrich the frame with object, overlay, broker-source, and chart-transform truth.
- 5 Decision modules derive market reality, regime, price location, scenario consensus, smart-money context, and timing readiness.
- 6 Model Council V3 evaluates the snapshot and produces a study packet or an executable packet candidate.
- 7 `packet_v3.py` builds and validates `PG_EXECUTION_PACKET_V3`.
- 8 The FastAPI app exposes latest study and execution packets.
- 9 `shooter.py` fetches the execution packet endpoint and rejects absent, stale, malformed, non-executable, or contradictory packets.
- 10 Shooter gate 1 requires a second live read. Gate 2 enforces trade discipline and cooldowns. Gate 3 confirms model council, side, timing sequence, calibration, and runtime integrity.
- 11 `ShooterActionSequencerV2` activates the broker window, sets time, performs final pre-click recheck, and clicks BUY or SELL only.
- 12 The runtime writes handshake, floating state, evidence, telemetry, traces, and outcome memory.

## Production Launch Setup

The project treats `launch_phoenixguard_live_ready.ps1` as the production launcher. It is referenced by `phoenixguard/V3_CANONICAL_MANIFEST.json` as the `FINAL_LIVE` entrypoint. The README documents the clean launch pattern:

```
Set-Location "c:\Users\thaba\OneDrive\Documents\The 808 Vision 2026\phoenixguard"
.\.venv\Scripts\Activate.ps1
.\launch_phoenixguard_live_ready.ps1
```

Read-only launch is available with:

```
.\launch_phoenixguard_live_ready.ps1 -DisableShooter
```

Before relaunching a strict runtime, the project expects V3 cleanup and integrity checks:

```
python .\tools\clean_v3_runtime_state.py --apply
python .\tools\runtime_trace_v3.py --base-url http://127.0.0.1:8793 --session pocket-live-8788 --timeout 20
python .\tools\trace_sequence_context_v3.py --base-url http://127.0.0.1:8793 --session pocket-live-8788 --timeout 20
python .\tools\verify_v3_integrity.py
```

## Repository Structure

| Path                | Meaning                                                                                                                                               |
|---------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------|
| phoenixguard/core   | Shared configuration, utility functions, and decision-state helpers.                                                                                  |
| phoenixguard/vision | Image preprocessing, broker source lock, chart segmentation, overlay schema, V3 overlay contract, transforms, rendering, and object registry support. |

| Path                    | Meaning                                                                                                                                                                            |
|-------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| phoenixguard/tracking   | V3 market object tracker and sequence-context bridge for overlays and registry state.                                                                                              |
| phoenixguard/mobile_api | FastAPI app, mobile service, observer service, continuous window tracker, live-state builder, dashboard, and real-time frontend sync.                                              |
| phoenixguard/decision   | Market reasoning, Model Council, RL, regression, ensemble, skill gates, scenarios, market memory, price location, regime, and outcome feedback.                                    |
| phoenixguard/execution  | Execution language, packet schema, governor, timing, sequence context, calibration manifest, shooter sequencing, shooter modes, and floating-state reducer.                        |
| phoenixguard/runtime    | Local ensemble runtime, model council daemon, adaptive runtime, LoRA adapters, cache V3, telemetry, certification, instrument context, security, and atomic performance utilities. |
| phoenixguard/memory     | Memory-bank ingest, vector search, style/trajectory features, and visual-play memory confirmation.                                                                                 |
| phoenixguard/simulation | Screenshot replay, paper execution, event backtesting, gym environment, overlay evaluation, decision replay, adversarial tests, and synthetic scenarios.                           |
| phoenixguard/training   | CV model training and sequence auxiliary-head support.                                                                                                                             |
| phoenixguard/voice      | Voice command parsing, local/remote tracker control, voice bundles, console UI, and local voice runtime.                                                                           |
| mobile/android          | Kotlin Android mobile observer client and UI model.                                                                                                                                |
| tools                   | Runtime tracing, certification, visual evidence capture, V3 cleanup, diagnostics, overlay validation, performance reports, and integrity checks.                                   |
| scripts                 | Dataset splitting, sequence teacher manifests, inference exports, signal replay, calibration manifest building, and contradiction queue export.                                    |
| docs                    | Architecture, frontend, tracker, share, scenario, vision, runtime, and deployment documentation.                                                                                   |

## Core Runtime Concepts

### Observation, Study, Execution

PhoenixGuard uses three separate authority levels:

- 1 Observation: raw perception, chart state, model scores, dashboard state, raw side, candidate side, and diagnostic gates.



- 1 Study: `STUDY_PACKET` from Model Council. It carries packet id, candidate context, score, threshold, blocked reason, next required condition, and promotion trace. It cannot execute.
- 1 Execution: `PG_EXECUTION_PACKET_V3`. It must contain `execution.enabled=true`, `execution.state=EXECUTABLE`, `execution.side`, `execution.expiry_seconds`, `execution.time_sequence`, runtime integrity, model health, instrument context, and Model Council final agreement.

## State Advancement

The live stack uses frame counters, capture counters, `state_version`, packet ids, TTL, hashes, heartbeat records, and cache metadata to prove that a decision belongs to the current live frame rather than stale memory or an old endpoint response.

## Side Semantics

The V3 language constitution separates side vocabulary:

- 1 `raw_side`: observation-level direction.
- 1 `candidate_side`: side under Model Council evaluation.
- 1 `final_side`: Model Council arbitration result.
- 1 `execution.side`: the only side that can reach broker action.

The packet validator requires final side and execution side to agree. Generic side is display shorthand only.

## Capture And Window Tracker Design

The continuous tracker is implemented mainly in `phoenixguard/mobile_api/window_tracker.py`. It is the largest runtime module because it combines window capture, source control, chart-study derivation, overlay rendering, session persistence, Model Council publication, and live dashboard payload construction.

Key design responsibilities:

- 1 Lists candidate windows and locks the selected broker or study surface.
- 1 Sets DPI awareness early on Windows so click coordinates and captures match the real UI.
- 1 Captures a full window or focus-region image.
- 1 Detects wrong-surface conditions and reacquires or blocks when the active surface is not the expected broker/chart.
- 1 Builds candle rows, chart statistics, global/local/impulse structure, support/resistance zones, projected sniper/trigger/target zones, SMC context, and timing payloads.
- 1 Publishes `session.json`, display state, preview images, event logs, latest chart/window artifacts, and live dashboard state.
- 1 Sends snapshots into `ModelCouncilV3`.
- 1 Exposes worker health, capture count, frame timing, state freshness, and study/execution packet endpoints through the API service.

## Tracker Techniques Used

- 1 Windows window enumeration and image capture.
- 1 DPI-aware coordinate handling.

- 1 Locked focus-region capture.
- 1 Broker-source fingerprinting and wrong-surface rejection.
- 1 Candle extraction and normalized chart geometry.
- 1 Historical structure segmentation.
- 1 Support/resistance mapping.
- 1 Smart-money concept tagging: order-block retests, fair-value gaps, liquidity sweeps/pools, market-structure shift, and entry/target levels.
- 1 Atomic session writes and display-state merge.
- 1 Adaptive capture intervals and worker watchdogs.
- 1 Visual overlay rendering for live dashboard and evidence images.

## Vision Layer

The vision layer contains image preprocessing, visual model orchestration, overlay contracts, and broker surface safety.

| File                                            | Role                                                                                                                                         |
|-------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------|
| phoenixguard/vision/preprocess.py               | Loads arbitrary image/file input, extracts price values, applies CLAHE, crops chart areas, normalizes input, and converts images to tensors. |
| phoenixguard/vision/multi_model_ensemble.py     | Provides model registry and multi-model detection/inference output structures for YOLO, ViT, and SAM-style backends.                         |
| phoenixguard/vision/broker_source_lock_v3.py    | Fingerprints broker viewport, pixels, and controls, classifies wrong surfaces, and builds broker-source lock status.                         |
| phoenixguard/vision/v3_chart_transform.py       | Maps normalized chart coordinates to chart image and screen coordinates.                                                                     |
| phoenixguard/vision/v3_overlay_contract.py      | Defines V3 overlay type, layer, coordinate-mode, and visibility contracts.                                                                   |
| phoenixguard/vision/overlay_geometry.py         | Normalizes and clips boxes, computes IoU/area/aspect, merges boxes, smooths overlays, and applies mode visibility.                           |
| phoenixguard/vision/overlay_layer_manager_v3.py | Resolves overlay layer order and view-mode behavior.                                                                                         |
| phoenixguard/vision/candle_snap.py              | Snaps candle geometry and validates candle overlap and metrics.                                                                              |
| phoenixguard/vision/chart_segmentation.py       | Segments chart regions, extracts chart area, and stores segmentation history.                                                                |
| phoenixguard/vision/renderer.py                 | Renders overlays on chart images.                                                                                                            |

## Vision Techniques Used

- 1 Classical image preprocessing: cropping, contrast enhancement, normalization, tensor conversion.
- 1 OCR-assisted and regex-assisted numeric extraction where relevant.
- 1 Box geometry hygiene: clipping, IoU, overlap ratio, structural anchoring, smoothing, and visibility rules.
- 1 Broker-surface fingerprinting using viewport and control evidence.
- 1 Overlay contract validation so dashboard visuals remain consistent across modes.
- 1 Multi-model ensemble hooks for YOLO, ViT, SAM-style segmentation, CLIP, DINOv2, SimCLR, SwAV, BYOL, MobileNetV3, and local specialist heads.

## Market Object Registry And Overlay Architecture

phoenixguard/tracking/market\_object\_tracker\_v3.py builds V3 market objects from session payloads and overlays. The registry gives the runtime a structured inventory of candles, zones, paths, labels, support/resistance, and sequence context.

The overlay system uses these protections:

- 1 Stable overlay ids and type names.
- 1 Known coordinate modes.
- 1 Layer ordering.
- 1 Visibility by mode.
- 1 Bounding box normalization.
- 1 Contract validation through tools/validate\_overlay\_contract\_v3.py.
- 1 Visual regression through dashboard and overlay tests.

This prevents overlay drift from becoming execution truth. Visual overlays explain the decision surface, but executable authority still comes only from the V3 packet.

## Memory System

The memory subsystem is implemented in phoenixguard/memory/memory\_ingest.py, phoenixguard/memory/memory\_features.py, and phoenixguard/memory/visual\_play\_memory\_bank.py.

Key capabilities:

- 1 Stores memory entries with images, labels, descriptions, style profiles, and trajectory signatures.
- 1 Embeds descriptions and optionally visual/context features.
- 1 Searches memory by vector similarity through an HNSW-style index.
- 1 Derives entry progression, style signatures, trajectory signatures, metric profiles, and late-interaction tokens.
- 1 Compares current chart behavior against recalled examples.
- 1 Provides transition probabilities and sequence context for decision modules.
- 1 Confirms visual-play memory matches before increasing confidence.

Memory is a contributor. It can boost or contextualize confidence, but it cannot create live execution authority on its own.

# Decision And Reasoning Layer

The decision package is intentionally multi-agent and multi-technique. It does not trust a single model output. Instead it combines statistical forecasting, heuristics, market reality checks, model-role votes, memory confirmation, and council promotion stages.

| Module                      | Responsibility                                                                                                                                         |
|-----------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------|
| model_council_v3.py         | Evaluates candidate sides, maturity stages, lane thresholds, timing, market context, runtime health, and publishes study/execution packet payloads.    |
| market_reality_engine.py    | Analyzes market trap, path risk, time-to-reward/invalidation, current candle contract, permission stack, and trade candidate queue.                    |
| market_intelligence_v3.py   | Scores angle dynamics, zone liquidity, historical pattern, opposing force, global structure, local microstructure, middle safety, and bad entry class. |
| decision_kernel.py          | Computes expected utility, target-before-invalidation style fields, belief state, and firewall advisory.                                               |
| price_location_engine_v3.py | Classifies current price location against global/local ranges and nearby zones.                                                                        |
| regime_engine_v3.py         | Reads trend, volatility, explicit regime hints, and candle-derived state.                                                                              |
| reasoning_arbitrator_v3.py  | Builds model role votes and detects bad-entry filters.                                                                                                 |
| ensemble.py                 | Fuses RL probabilities, memory similarity, forecast interval constraints, and gate diagnostics.                                                        |
| regression_module.py        | Provides Chronos/image-fusion forecasts, conformal intervals, and forecast routing.                                                                    |
| rl_module.py                | Provides GRPO-style policy head, inference, reward calculation, online update, feedback recording, and recall-driven updates.                          |
| skill_gates.py              | Runs 12 diagnostic curriculum gates and router weights.                                                                                                |
| a_star_scenarios.py         | Uses A-star scenario search to predict future candle paths.                                                                                            |
| scenario_integration.py     | Converts chart state into scenarios, ranks scenario agreement, and converts scenario output to paint layers.                                           |
| outcome_feedback_v3.py      | Builds and logs outcome records and updates pair behavior profiles.                                                                                    |

## Model Council V3

Model Council V3 is the live arbitration layer. It scores BUY and SELL candidates from the current snapshot, attaches market intelligence, evaluates maturity, selects an execution lane, computes release requirements, and returns either a non-executable study packet or an executable packet candidate.

The maturity ladder is:

- 1 OBSERVATION
- 2 HYPOTHESIS
- 3 CONTEXT\_CONFIRMATION
- 4 ZONE\_QUALIFICATION
- 5 TIMING\_READINESS
- 6 EXECUTION\_MATURITY
- 7 EXECUTABLE\_PACKET

Default execution lane thresholds include:

| Lane                         | Score Threshold |
|------------------------------|-----------------|
| HIGH_FREQUENCY_TWO_CANDLE    | 0.50            |
| SNIPER_ZONE_ENTRY            | 0.70            |
| FAILED_RETEST_ENTRY          | 0.72            |
| LOCAL_BREAKDOWN_CONTINUATION | 0.74            |
| HISTORY_MATCHED_CONTINUATION | 0.76            |
| MOMENTUM_ACCEPTANCE_ENTRY    | 0.82            |

## Skill Gates

The 12-gate curriculum is diagnostic in the live path. It can explain, warn, and provide analytics, but it cannot directly select side, create HOLD, inflate confidence, choose expiry, veto a tracker-backed setup, or trigger the shooter.

The gates cover:

- 1 Probability and conformal calibration.
- 2 Discrete finite-state progression.
- 3 Algorithmic heap signal ranking.
- 4 Meta stacking.
- 5 Context retrieval.
- 6 Operational stability.
- 7 UI analytics.
- 8 Meta constraints.
- 9 Regression error estimation.
- 10 Knowledge representation and ontology coherence.
- 11 Formal automata state progression.
- 12 Predictive analytics fusion.

## Scenario And Simulation Reasoning

PhoenixGuard includes A-star candle scenario search, synthetic market scenarios, adversarial trap cases, screenshot replay, paper execution, event candle backtesting, and overlay metric evaluation. These techniques support offline validation, replay debugging, and training feedback. They are not direct live-click authorities.

## Execution Packet V3 Contract

`phoenixguard/execution/packet_v3.py` defines the canonical execution packet schema and validation helpers.

An executable packet must include:

- 1 `schema_version == PG_EXECUTION_PACKET_V3`
- 1 Non-empty `packet_id`
- 1 Session, symbol, timeframe, frame id, capture count, and state version
- 1 `execution.enabled == true`
- 1 `execution.state == EXECUTABLE`
- 1 `execution.side` equal to Model Council `final_side`
- 1 Positive `execution.expiry_seconds`
- 1 `execution.time_sequence.target_seconds` equal to `execution.expiry_seconds`
- 1 Live integrity hashes and counters
- 1 Runtime model health
- 1 Instrument context and symbol context
- 1 Sequence context
- 1 Current TTL and valid-until timestamp
- 1 Cache state compatible with live execution

The validator rejects stale packets, side mismatch, missing packet identity, raw action payloads, invalid schema versions, missing model health, missing sequence context, bad instrument context, runtime integrity failures, and ambiguous or contradictory timing fields.

## Shooter And Broker Action Design

`shooter.py` is the live execution consumer. It is deliberately separate from the tracker/API process. The tracker can publish state and packets; the shooter alone performs calibrated broker actions.

Key shooter stages:

- 1 Finds and activates the broker window.
- 1 Loads `808_shooter_boxes.json`.
- 1 Resolves reachable API base URL.
- 1 Fetches `/v1/mobile/model-council/.../execution/latest`.
- 1 Rejects missing packets and displays study wait state when only study packets exist.
- 1 Validates V3 packet schema and runtime integrity.
- 1 Performs a second live read to confirm counters advanced.
- 1 Applies trade discipline, duplicate-signal protection, cooldowns, and packet consumption state.

- 1 Validates calibration targets and broker layout.
- 1 Confirms expiry/time sequence.
- 1 Uses ShooterActionSequencerV2 for window activation, time setting, final pre-click recheck, and side click.
- 1 Writes shooter handshake and action evidence.

## Calibrated Targets

The canonical required targets are listed in `phoenixguard/V3_CANONICAL_MANIFEST.json`:

- 1 `broker_screen`
- 1 `time_button`
- 1 `hourly_input`
- 1 `minute_input`
- 1 `buy_icon`
- 1 `sell_icon`
- 1 `final_screen`

The system preserves the broker amount. Amount calibration and amount editing are outside the live control loop.

## Shooter Techniques Used

- 1 Windows API window discovery and foreground activation.
- 1 Relative-to-absolute coordinate conversion.
- 1 Broker timing profile loaded from `config/shooter_broker_timing_profile.json`.
- 1 Time sequence typing and arrow fallback support.
- 1 OCR-assisted visible time checks where available.
- 1 Final pre-click confirmation.
- 1 Evidence screenshot capture with target annotation.
- 1 Paper, dry-run, calibration-test, live-disabled, and live-ready modes.
- 1 Explicit environment gating through `PHOENIXGUARD_ALLOW_LIVE_BROKER_CLICKS`.

## API And Dashboard Layer

`phoenixguard/mobile_api/app.py` creates the FastAPI app. It exposes health, live state, model council, tracker, observer, dashboard, artifact, visual health, registry, performance, voice, and streaming endpoints.

Important endpoint groups:

| Endpoint Group                               | Purpose                                                              |
|----------------------------------------------|----------------------------------------------------------------------|
| <code>/v1/mobile/health</code>               | API health check.                                                    |
| <code>/v1/mobile/live/state/v3</code>        | Compact or full live-state payload for dashboards and frontend sync. |
| <code>/v1/mobile/model-council/latest</code> | Latest Model Council state for diagnostics.                          |

| Endpoint Group                                         | Purpose                                                                                                    |
|--------------------------------------------------------|------------------------------------------------------------------------------------------------------------|
| /v1/mobile/model-council/study/latest                  | Latest non-executable study packet.                                                                        |
| /v1/mobile/model-council/execution/latest              | Latest executable V3 packet or 404 when not executable.                                                    |
| /v1/mobile/floating/state                              | FloatingStateV2 truth for operator display.                                                                |
| /v1/mobile/shooter/handshake                           | Latest shooter handshake and wait/action state.                                                            |
| /v1/mobile/runtime/trace/v3                            | End-to-end runtime trace across tracker, council, packet, shooter, floating state, cache, and calibration. |
| /v1/mobile/window-tracker/sessions                     | Create, list, inspect, start, stop, and control tracker sessions.                                          |
| /v1/mobile/window-tracker/dashboard/{session_id}       | Live dashboard.                                                                                            |
| /v1/mobile/window-tracker/sessions/{session_id}/stream | Server-sent event stream for session updates.                                                              |
| /v1/mobile/visual/health/v3                            | Visual health and overlay/frontend truth checks.                                                           |
| /v1/mobile/registry/sessions/{session_id}/active       | Current active market object registry.                                                                     |

The dashboard consumes tracker state and shows market context, overlays, study maps, timing lock rows, packet state, shooter state, and visual health. It is diagnostic and operational, not direct execution authority.

## Mobile Observer

The mobile side has two forms:

- 1 phoenixguard/mobile\_api/service.py and observer.py handle manual/mobile job uploads and observer bundles.
- 1 mobile/android contains the Kotlin Android client structure, including MainActivity.kt, model classes, view model, Compose screen, theme, and Android manifest.

Observer latest signals remain legacy diagnostics under V3. They must not become live execution authority.

## Voice Control

The voice package provides a local and remote command layer for runtime control and status.

| File                          | Responsibility                                                                                        |
|-------------------------------|-------------------------------------------------------------------------------------------------------|
| phoenixguard/voice/intents.py | Parses voice commands, extracts timing, blocks sensitive disclosure, and publishes command catalog.   |
| phoenixguard/voice/router.py  | Registers and resolves voice commands.                                                                |
| phoenixguard/voice/control.py | Applies voice preferences, state updates, command execution, status/help responses, and console HTML. |



| File                          | Responsibility                                                                                      |
|-------------------------------|-----------------------------------------------------------------------------------------------------|
| phoenixguard/voice/live.py    | Controls local tracker start/stop/capture/interval and builds market context from tracker sessions. |
| phoenixguard/voice/remote.py  | Controls a remote tracker through API calls.                                                        |
| phoenixguard/voice/bundles.py | Resolves local model bundles and validates bundle manifests.                                        |
| phoenixguard/voice/agent.py   | Provides local voice agent runtime and stack status.                                                |

Voice commands control workflow and diagnostics. They do not bypass packet validation or shooter gates.

## Training And Local Model Runtime

PhoenixGuard uses a local model training/runtime stack rather than depending only on external inference.

phoenixguard/training/ensemble\_cv\_models.py includes:

- 1 Model train configs.
- 1 Replay samples and continual training state.
- 1 Focal cross entropy.
- 1 Sequence auxiliary head.
- 1 Chart image dataset.
- 1 Transform builders for basic, DINOv2, and self-supervised boost views.
- 1 Triplet loss, probability-gap penalty, feature pooling, and backbone feature forwarding.

phoenixguard/runtime/local\_ensemble\_runtime.py loads saved local CV bundles and routes prediction among model roles:

| Model       | Runtime Role         |
|-------------|----------------------|
| mobilenetv3 | execution specialist |
| clip        | buy specialist       |
| simclr      | sell specialist      |
| swav        | generalist           |
| dinov2      | structure specialist |
| byol        | buy specialist       |

The runtime supports CPU/GPU model selection, saved bundle discovery, ONNX/export metadata paths, temperatures, thresholds, adapter activation, and cache-aware prediction.

## Continual Adaptation And Adapters

The adaptive runtime and adapter system support test-time view selection, open-set detection, context keys, and LoRA-style adapters.

Key files:

```
1 phoenixguard/runtime/adaptive_runtime.py
1 phoenixguard/runtime/continual_adapters.py
1 scripts/export_inference_bundles.py
1 train_finetune.py
1 train_finetune_dinov2_only.py
1 train_finetune_remaining.py
1 train_sequence_aware_all.py
```

Techniques include low-rank linear/conv adapters, adapter bank summaries, active adapter selection, replay sample curation, sequence teacher manifests, and inference export metadata.

## Runtime Integrity And Observability

Runtime integrity is enforced by multiple modules:

| Module                                          | Integrity Responsibility                                                                                                                                             |
|-------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| phoenixguard/runtime/cache_v3.py                | Validates cache records, attaches cache metadata, verifies study packet current-state use, validates execution packet cache state, and decides packet executability. |
| phoenixguard/runtime/instrument_context.py      | Builds and validates symbol, timeframe, viewport, broker surface, and instrument identity context.                                                                   |
| phoenixguard/runtime/realtime_performance_v3.py | Atomic session writes, freshness validators, frame timing trace, capture watchdog, latest frame buffer, and performance trace.                                       |
| phoenixguard/runtime/observability_v3.py        | Compute usage, telemetry, Model Council health, intelligence health, forensic decision log, bad-entry replay, and paper-mode records.                                |
| phoenixguard/runtime/certification_v3.py        | Certification gate results, session freshness, capture worker health, and report writing.                                                                            |
| phoenixguard/tracing.py                         | OpenTelemetry setup and FastAPI instrumentation.                                                                                                                     |

The default tracing endpoint is `http://localhost:4318/v1/traces`. Tracing can be disabled with `PHOENIXGUARD_TRACING_DISABLED=1`.

## Security And State Protection

phoenixguard/runtime/security.py provides:

```
1 Fernet key derivation.
1 File encryption/decryption.
1 Secure delete and tensor-like memory wipe helpers.
1 Encrypted preference store.
```

1 Unavailable fallback preference store.

Execution safety is also a security concern. The shooter requires explicit live-click arming, validates packet identity and calibration, and preserves broker amount. Legacy raw execution paths are listed as forbidden in the V3 canonical manifest.

## Simulation, Backtesting, And Replay

PhoenixGuard includes extensive simulation support:

| Area                | Files                                                          | Purpose                                                                                                                   |
|---------------------|----------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------|
| Screenshot replay   | phoenixguard/simulation/screenshot_replay/*                    | Loads captured frames, controls replay speed, publishes replay packets, records replay metrics, and runs replay sessions. |
| Paper execution     | phoenixguard/simulation/paper_execution/*                      | Records executable packets and rehearses broker-demo behavior without live broker authority.                              |
| Event backtesting   | phoenixguard/simulation/event_backtesting/candle_backtester.py | Runs event candle backtests and parameter sweeps.                                                                         |
| Adversarial tests   | phoenixguard/simulation/adversarial_tests/*                    | Generates fake breakout, range chop, opposing force, and steep impulse cases.                                             |
| Synthetic scenarios | phoenixguard/simulation/synthetic_scenarios/*                  | Generates synthetic market scenario suites and labels expected decisions.                                                 |
| Overlay eval        | phoenixguard/simulation/overlay_eval/*                         | Scores box metrics, temporal jitter, label clutter, and zone anchoring.                                                   |
| Decision replay     | phoenixguard/simulation/decision_replay/*                      | Replays council decisions and records agent votes/maturity stages.                                                        |

This lets the system improve logic without relying only on live sessions.

## Testing And Certification Matrix

The test suite is broad and aligned with the architecture.

| Test Area                | Representative Tests                                                                                                       |
|--------------------------|----------------------------------------------------------------------------------------------------------------------------|
| V3 schema and language   | tests/test_execution_packet_schema_v3.py, tests/test_v3_language_contracts.py, tests/test_v3_integrity.py                  |
| Tracker and window state | tests/test_window_tracker_service.py, tests/test_window_tracker_payload_and_projection.py, tests/test_tracker_bootstrap.py |

| Test Area                             | Representative Tests                                                                                                                                                             |
|---------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Model Council and market intelligence | tests/test_model_council_v3.py,<br>tests/test_market_reality_engine.py,<br>tests/test_market_intelligence_v3.py                                                                  |
| Shooter and execution                 | tests/test_shooter_v3_runtime.py,<br>tests/test_shooter_action_sequencer.py,<br>tests/test_execution_governor.py,<br>tests/test_execution_constitution.py                        |
| Visual and overlay                    | tests/test_visual_health_and_overlay_migration.py, tests/test_overlay_precision_v3.py,<br>tests/test_v3_overlay_contract.py,<br>tests/test_visual_regression.py                  |
| Runtime performance                   | tests/test_realtime_performance_v3.py,<br>tests/test_cache_observability_v3.py,<br>tests/test_runtime_telemetry_v3.py,<br>tests/test_session_atomic_v3.py                        |
| Memory, RL, and sequence              | tests/test_memory_sequence_retrieval.py,<br>tests/test_rl_runtime_integration.py,<br>tests/test_sequence_projection.py,<br>tests/test_sequence_teacher_manifest.py               |
| Simulation                            | tests/test_simulation_replay_stack.py,<br>tests/test_simulation_paper_execution.py,<br>tests/test_simulation_event_backtesting.py,<br>tests/test_adversarial_market_simulator.py |
| Voice and mobile API                  | tests/test_voice_api.py,<br>tests/test_voice_command_router.py,<br>tests/test_mobile_api_service.py,<br>tests/test_mobile_observer_service.py                                    |

Certification tools under `tools/` cover API stability, process topology, dashboard hydration, capture worker, live speed, model warm state, broker source lock, overlay truth, shooter persistence, wrong-surface rejection, and V3 burn-in.

## Deployment And Sharing

Deployment support is concentrated in `deploy/windows` and `deploy/cloudflare`.

Windows VM scripts:

```
1 Start-PhoenixGuardVmMonitor.ps1
1 Start-PhoenixGuardVmShare.ps1
1 Start-PhoenixGuardQuickTunnel.ps1
1 Stop-PhoenixGuardQuickTunnel.ps1
1 Register-PhoenixGuardWatchdogTask.ps1
1 Register-PhoenixGuardVmMonitorTask.ps1
```

```

1 Register-PhoenixGuardShareTask.ps1
1 Setup-PhoenixGuardCloudflare.ps1
1 Install-CloudflaredTunnel.ps1

```

Cloudflare support:

```

1 deploy/cloudflare/phoenixguard-share.example.yml
1 docs/share/WORLDWIDE_SHARE.md
1 docs/share/FRONTEND_SHARE_UPGRADE_BLUEPRINT.md

```

The quick tunnel path can publish the local API/dashboard when public browser access is required. It does not replace local packet validation or shooter safety.

## Important Data Artifacts

| Artifact                                                          | Purpose                                                              |
|-------------------------------------------------------------------|----------------------------------------------------------------------|
| 808_shooter_boxes.json                                            | Calibrated broker target positions.                                  |
| user_calibration_manifest.json                                    | User calibration manifest.                                           |
| config/shooter_broker_timing_profile.json                         | Timing profile for broker action sequencing.                         |
| .codex_runtime/tracker_status.json                                | Tracker status.                                                      |
| .codex_runtime/vm_monitor_status.json                             | VM monitor status.                                                   |
| .codex_runtime/action_evidence                                    | Shooter evidence artifacts.                                          |
| data/mobile_api/window_tracker/sessions/<session_id>/session.json | Tracker session state.                                               |
| data/mobile_api/observer/sessions/<session_id>                    | Observer session bundles and latest signal diagnostics.              |
| tests/fixtures/visual_regression                                  | Visual regression baselines.                                         |
| reports                                                           | Launch, trace, validation, and certification reports when generated. |
| memory_bank and 808 Memory                                        | Local memory examples and training context.                          |
| models                                                            | Saved local model bundles and exports.                               |

## Operator Safety Rules

The live shooter is capable of real broker clicks. Strict operation requires:

- 1 Run integrity checks before live signal mode.
- 1 Keep the broker amount manually controlled and unchanged by PhoenixGuard.
- 1 Use -DisableShooter for read-only monitoring.
- 1 Set PHOENIXGUARD\_ALLOW\_LIVE\_BROKER\_CLICKS=1 only when deliberately entering live-ready mode.

- 1 Validate calibrated target boxes against the current broker layout.
- 1 Confirm tracker source lock and wrong-surface rejection.
- 1 Treat observer signals and dashboard fields as diagnostics only.
- 1 Never re-enable legacy raw action execution as live authority.

Live signal mode should follow the README pattern:

```
$env:PHOENIXGUARD_ALLOW_LIVE_BROKER_CLICKS="1"
python shooter.py signal --session-id pocket-live-8788 --base-url http://127.0.0.1:8793 --poll 0.05 --max-signal-age 8 --preferre
```

## Failure Handling And Diagnostics

The active execution path document defines key diagnosis cases:

| Tracker        | Council            | Study Packet   | Execution Packet | Meaning                             |
|----------------|--------------------|----------------|------------------|-------------------------------------|
| current        | WATCHING/PREPARING | present        | missing          | Council has not promoted yet.       |
| current        | EXECUTABLE         | present        | missing          | Publisher or endpoint wiring fault. |
| current        | EXECUTABLE         | present        | present          | Shooter should reach V3 gate 1.     |
| endpoint error | endpoint error     | endpoint error | endpoint error   | PhoenixGuard API process is down.   |

Primary diagnostic command:

```
python tools\diagnose_v3_execution_path.py --session pocket-live-8788 --base-url http://127.0.0.1:8793
```

Other useful tools:

```
python tools\runtime_trace_v3.py --base-url http://127.0.0.1:8793 --session pocket-live-8788
python tools\trace_backend_frontend_v3.py --base-url http://127.0.0.1:8793 --session pocket-live-8788
python tools\trace_overlay_source_v3.py --base-url http://127.0.0.1:8793 --session pocket-live-8788
python tools\trace_frame_timing_v3.py --base-url http://127.0.0.1:8793 --session pocket-live-8788
python tools\verify_v3_integrity.py
```

## Design Principles Used

PhoenixGuard's design uses these engineering principles:

- 1 Separation of observation, study, arbitration, and execution authority.
- 1 Local-first runtime and model assets for privacy and repeatability.
- 1 Explicit schema contracts instead of loosely interpreted signals.
- 1 Atomic state writes for dashboard and session consistency.
- 1 Freshness proofs through counters, hashes, TTL, and state versions.
- 1 Defense in depth before live broker interaction.
- 1 Diagnostics-first dashboards that reveal why a setup is waiting.
- 1 Runtime traceability from frame capture to shooter handshake.

- 1 Memory as context, not authority.
- 1 Visual overlays as explanation, not execution.
- 1 Continuous replay/backtest/certification loops for quality control.

## End-To-End Build Explanation

The project was built as a Python-centered workstation with a Windows-focused execution layer and a FastAPI service boundary.

At the bottom, the system uses local files, model bundles, calibrated screen coordinates, session JSON, screenshots, and runtime manifests. Above that, the tracker captures a locked window and builds a normalized session state. Vision modules provide image and overlay truth. Memory modules add historical similarity and sequence comparisons. Decision modules convert the snapshot into structured market reasoning. Model Council promotes or blocks the setup. Packet V3 formalizes authority. FastAPI exposes only the current live state, study state, and executable packets. Shooter runs as a separate process so the executor has an independent validation pass.

The frontend and mobile layers are consumers of this state. They are intentionally not trusted to decide execution. The dashboard can show exactly what the backend knows, and the floating state can report what the operator needs, but the click path remains packet-driven and calibrated.

## Extension Points

The most important safe extension points are:

- 1 Add new visual models through `phoenixguard/runtime/local_ensemble_runtime.py` and training/export scripts.
- 1 Add new overlay types only through `v3_overlay_contract.py` and validation tests.
- 1 Add new Model Council evidence as diagnostic contributors unless packet schema changes require explicit V3 language updates.
- 1 Add new execution lanes by defining threshold, timing behavior, lane release requirements, and tests.
- 1 Add new API views as read-only consumers unless they expose existing validated packet state.
- 1 Add new simulation cases in `phoenixguard/simulation/adversarial_tests` or `synthetic_scenarios`.
- 1 Add new certification tools in `tools/` and link them to `verify_v3_integrity.py` where appropriate.

Unsafe extension points include direct action execution, raw observer signal execution, dashboard-triggered broker clicks, packet TTL bypasses, amount editing, and any shortcut that lets Model Council `final_side` bypass `execution.side`.

## Glossary

| Term                   | Meaning                                                      |
|------------------------|--------------------------------------------------------------|
| FINAL_LIVE             | Canonical production runtime profile.                        |
| STUDY_PACKET           | Non-executable packet describing a forming or blocked setup. |
| PG_EXECUTION_PACKET_V3 | Only schema allowed to authorize shooter execution.          |

| Term               | Meaning                                                                                     |
|--------------------|---------------------------------------------------------------------------------------------|
| FloatingStateV2    | Operator-facing truth state for current runtime and shooter status.                         |
| Model Council V3   | Arbitration layer that promotes candidates and publishes study/execution state.             |
| execution.side     | Only action side that can reach the shooter.                                                |
| final_side         | Council arbitration result; must match execution.side but cannot execute alone.             |
| raw_side           | Observation-level side from tracker/model/dashboard evidence.                               |
| candidate_side     | Side being evaluated by the council.                                                        |
| runtime_integrity  | Freshness, health, identity, cache, and live-state proof.                                   |
| instrument_context | Symbol, timeframe, viewport, and broker-surface identity lock.                              |
| time_sequence      | Explicit expiry/time-setting sequence for shooter.                                          |
| source lock        | Proof that the captured surface is the intended broker/chart, not a dashboard or wrong app. |
| overlay contract   | Rules for valid overlay types, coordinates, layers, ids, and modes.                         |

## Final Blueprint Summary

PhoenixGuard is a layered V3 architecture for live chart intelligence and controlled execution. It combines computer vision, chart geometry, memory retrieval, model ensembles, reinforcement learning, conformal/regression forecasting, scenario search, market-reality checks, smart-money context, schema validation, runtime telemetry, and calibrated UI automation.

The design is strongest where it is strict: the system does not let a model, dashboard, raw side, or old signal directly click. It requires a current frame, source lock, Model Council maturity, executable V3 packet, cache/freshness proof, instrument identity, valid timing sequence, calibrated targets, shooter discipline gates, and final pre-click confirmation. This is the core blueprint of how PhoenixGuard is built, what it does, how it does it, and why the architecture is shaped the way it is.